# Cahier du participant : Creer et deployer un chatbot autonome avec Ollama et Docker

Bienvenue dans ce laboratoire pratique. Vous allez apprendre a emballer (“wrapper”) un modele d’intelligence artificielle local et une interface Web en JavaScript dans un conteneur Docker, puis a deployer le tout gratuitement sur la plateforme Render.

---

## 1. Objectifs du laboratoire

A la fin de cet atelier, vous serez en mesure de :

- Comprendre comment un modele de langage (LLM) fonctionne a l’interieur d’un conteneur Linux.
- Construire une image Docker complete combinant le moteur Ollama et un serveur Node.js.
- Tester et valider votre conteneur localement sur votre poste de travail.
- Publier votre code source sur GitHub.
- Mettre en ligne votre application sur Render et observer son comportement en production.

---

## 2. Prerequis necessaires

Avant de commencer, assurez-vous d’avoir :

1. **Docker Desktop** (ou un moteur Docker compatible) installe et demarre sur votre ordinateur.
   - *Pour les utilisateurs de Windows :* Assurez-vous que Docker Desktop utilise le moteur **WSL 2** (dans Parametres > General > Use the WSL 2 based engine). Utilisez le terminal **PowerShell** ou **Windows Terminal** pour executer les commandes.
2. **Git** installe et configure avec votre cle SSH ou votre jeton d’acces.
3. Un compte gratuit sur **GitHub** (https://github.com).
4. Un compte gratuit sur **Render** (https://render.com).

*Remarque pour Windows :* Le depot comprend un fichier `.gitattributes` qui protege les scripts shell contre les conversions de fins de ligne Windows (CRLF). Vos scripts demeureront au format Unix (LF) compatible avec Linux.

---

## 3. Structure du projet

Voici l’arborescence des fichiers du projet qui vous sont fournis :

```
stf-docker/
├── Dockerfile              # Instructions pour fabriquer l'image Linux
├── entrypoint.sh           # Script qui demarre Ollama puis le serveur Node.js
├── package.json            # Declaration des dependances Node.js (Express)
├── server.js               # Serveur Web faisant le pont avec l'API d'Ollama
├── docker-compose.yml      # Raccourci pour tester le conteneur en local
├── .dockerignore           # Fichiers a exclure de la construction Docker
└── public/
    ├── index.html          # Page d'accueil de la salle de discussion
    ├── style.css           # Feuilles de style de l'interface
    └── app.js              # Code client pour l'envoi et la reception continue
```

---

## 4. Etape 1 : Comprendre les composants du projet

Prenez quelques minutes pour ouvrir les fichiers suivants et observer leur contenu :

### A. Le fichier `server.js`

- Observez la constante `MODEL_NAME = 'qwen2.5:0.5b'`. C'est le nom du modele tres leger que nous utilisons pour respecter les quotas de memoire vive.
- Observez la constante `OLLAMA_HOST = 'http://127.0.0.1:11434'`. Le serveur Node.js communique en reseau local direct avec Ollama puisque les deux tournent dans la meme machine virtuelle.
- Remarquez l'option `num_ctx: 1024` dans la requete envoyee a Ollama. Elle limite la fenetre de contexte pour empecher l'explosion de la memoire vive.

### B. Le fichier `Dockerfile`

- Ce fichier installe Ubuntu 22.04, Node.js 20 et Ollama.
- Remarquez l'etape d'inclusion prealable (le "Bake") :

  ```
  RUN ollama serve & ... && ollama pull qwen2.5:0.5b
  ```

  Sans cette ligne, votre conteneur devrait re-telecharger les 400 Mo du modele a chaque fois qu'il demarre sur Render.

### C. Le fichier `entrypoint.sh`

- Ce script est execute a chaque lancement du conteneur.
- Il lance d'abord `ollama serve &` en arriere-plan, verifie a l'aide de curl que le service est fonctionnel, puis lance l'application Node.js avec `exec node server.js`.

---

## 5. Etape 2 : Construire et tester le conteneur en local

Avant de tenter un deploiement sur Internet, nous devons toujours valider le bon fonctionnement de notre conteneur localement.

### Action 1 : Ouvrir un terminal dans le dossier du projet

Assurez-vous que votre invite de commande se trouve a la racine du projet :

```
pwd
```

### Action 2 : Construire l'image Docker

Lancez la commande de fabrication :

```
docker build -t mini-chatbot-ollama .
```

*Remarque :* La premiere construction peut prendre entre 3 et 5 minutes selon la vitesse de votre connexion Internet, car Docker doit installer les outils et telecharger les poids du modele Qwen 2.5 (environ 400 Mo).

### Action 3 : Demarrer le conteneur localement

Une fois la construction terminee, lancez le conteneur :

```
docker run -p 3000:3000 --name mon-chatbot-test --rm mini-chatbot-ollama
```

### Action 4 : Tester l'application

1. Ouvrez votre navigateur Web a l'adresse : `http://localhost:3000`.
2. Regardez la pastille en haut a droite : elle doit passer au vert et indiquer "Modele qwen2.5:0.5b pret".
3. Posez une question simple au modele, par exemple : "Qui a decouvert le fleuve Saint-Laurent ?".
4. Observez la generation de la reponse qui s'affiche mot par mot.
5. Une fois le test reussi, arretez le conteneur dans votre terminal en appuyant sur `Ctrl + C`.

---

## 6. Etape 3 : Publier votre code sur GitHub

Render a besoin de lire votre code a partir d'un depot Git pour pouvoir construire votre conteneur.

1. Connectez-vous sur votre compte **GitHub**.
2. Creez un nouveau depot public nomme `mini-chatbot-ollama` (ne cochez pas l'option pour ajouter un README ou un .gitignore, nous avons deja les fichiers).
3. Dans votre terminal local, initialisez Git et liez votre depot :

```
git init
git add .
git commit -m "Version initiale du chatbot avec Ollama et Docker"
git branch -M main
git remote add origin https://github.com/<votre-nom-utilisateur>/mini-chatbot-ollama.git
git push -u origin main
```

---

## 7. Etape 4 : Deployer l'application sur Render

1. Rendez-vous sur votre tableau de bord **Render** (https://dashboard.render.com).
2. Cliquez sur le bouton **New +** en haut a droite, puis choisissez **Web Service**.
3. Associez votre compte GitHub si ce n'est pas deja fait, et choisissez le depot `mini-chatbot-ollama`.
4. Remplissez les parametres de configuration comme suit :
   - **Name** : `mon-chatbot-ia` (ou le nom de votre choix).
   - **Region** : Choisissez la region la plus proche (ex. *Ohio (US East)* ou *Frankfurt (EU Central)*).
   - **Environment** : Choisissez **Docker** (Render detectera automatiquement votre `Dockerfile`).
   - **Branch** : `main`.
   - **Instance Type** : Selectionnez **Free (0$ / month)**.
5. Cliquez sur le bouton **Deploy Web Service** en bas de la page.

---

## 8. Etape 5 : Observer le deploiement et les journaux

1. Pendant le deploiement, observez l'onglet **Logs** :
   - Vous verrez Render executer chaque ligne de votre `Dockerfile`.
   - Vous constaterez l'installation d'Ollama et la confirmation du telechargement du modele.
2. Lorsque les messages suivants apparaissent :

   ```
   Lancement du service Ollama...
   Ollama est operationnel.
   Serveur web demarre sur le port ...
   ```

   Votre conteneur est officiellement en ligne.
3. Cliquez sur l'adresse URL generee par Render en haut de la page (du type `https://mon-chatbot-ia.onrender.com`).
4. Testez votre chatbot directement sur Internet depuis votre telephone ou votre ordinateur.

---

## 9. Les contraintes du plan gratuit a retenir

Prenez le temps d'observer le comportement de votre service en ligne :

1. **La mise en veille automatique :**
   - Apres 15 minutes d'inactivite, Render met votre conteneur en sommeil pour economiser les ressources.
   - La prochaine fois que vous ouvrirez l'URL, la page mettra environ 50 a 80 secondes a repondre. C'est ce qu'on appelle un *demarrage a froid* ("cold start").
2. **La memoire vive stricte de 512 Mo :**
   - Si vous essayez de remplacer le modele `qwen2.5:0.5b` par un modele comme `mistral:7b` ou `llama3:8b`, le conteneur s'eteindra immediatement avec un code d'erreur 137 ("OOMKilled").
3. **Le processeur partage :**
   - En l'absence de carte graphique (GPU), la generation de texte est un peu plus lente que sur les grands services commerciaux. Le protocole de flux continu (SSE) que nous avons programme permet de masquer ce delai en montrant les mots aussitot generes.

---

## 10. Defis optionnels pour aller plus loin

Pour consolider vos acquis, voici deux ameliorations possibles :

### Defi 1 : Personnaliser l'identite du modele

Dans `server.js`, avant d'envoyer les messages a Ollama, inserez un message systeme au debut de la liste des messages pour lui donner un role :

```
const messageSysteme = {
  role: 'system',
  content: 'Tu es un tuteur amical specialiste de l'histoire du Quebec. Reponds de facon concise et
        encourageante.'
};
```

Faites un `git commit` puis un `git push`, et observez Render reconstruire automatiquement votre application.

### Defi 2 : Ajouter un bouton pour vider la discussion

Dans `public/index.html` et `public/app.js`, ajoutez un bouton permettant a l'utilisateur de reinitialiser le tableau `historiqueMessages` et de nettoyer la zone d'affichage.